Lab 1: Introduction to Python

Overview

The primary objective of Lab 1 is to provide students with a comprehensive introduction to Python programming and its essential libraries, laying the groundwork for their future EE 347 journey. During this lab, students will gain familiarity with Python by setting up and configuring their development environment, including the installation of Python libraries. Through guided exercises, they will learn to utilize some of the most important libraries for EE 347: NumPy, SymPy, and SciPy.

Prerequisites for effectively completing this lab:

- Basic understanding of programming concepts
- Basic Familiarity with Python programming language

Materials:

- Laptop (bring your own)

Part I: Introduction to Python for Robotics **3**
Deliverable(s): 3
Part II: Demo **3**
Python setup demo 3
Scientific computing in Python demo 5
NumPy - Numerical Computing with Python 5
SciPy 6
SymPy - Symbolic Mathematics in Python 7
Kinematics Example 7
Kinematics demo code 1: 8
Kinematics demo code 2: 9
Introduction to Elephant Robotics **13**
Grading Policy: **14**
Lab Weightings: 14
Total Lab Score Calculation: 14
Lab 1: Points for Preparation 14
Lab 1 Completion Verification: 14
Lab 1 Grading: 15
Lab Participation and Engagement: 15
Consultation and Clarifications: 15

Part I: Introduction to Python for Robotics

Python has emerged as a popular programming language in the field of robotics, offering a versatile and powerful platform for designing, simulating, and controlling robotic systems. Python's libraries, such as NumPy, SymPy, and SciPy, provide essential tools for performing kinematic calculations efficiently.

Task:

- ☐ Install Python (macOS Instructions / windows (x86) instructions) - **includes instructions to set up Python Virtual Environment**
- ☐ Install Pycharm (Instructions)
  - ☐ Note: Alternative IDEs or editors such as VSCode are okay, but it is the student's responsibility to ensure that it works with the lab requirements

Deliverable(s):

- ☐ Live Demo: Demo the part II of this lab to the teaching team in person
  - ☐ Python setup demo
  - ☐ Numpy, Scipy, and Sympy demo
  - ☐ Kinematics demo 1 and 2
  - ☐ Individual Demo Quiz

Part II: Demo

The provided demo codes serve two primary purposes:

1. Environment Verification: To ensure your development environment is set up correctly.
2. Instructional Guidance: To teach you how to effectively use the required libraries, illustrate essential Python syntax, and provide sample code that will be useful for solving future lab assignments.

It is recommended that you do not merely copy and paste the code. Instead, take the time to thoroughly analyze it. Please note that some runtime errors are **intentional** and are designed to simulate common issues that may arise due to missing steps or misconfiguration. These errors are valuable exercises in troubleshooting and problem-solving. Treat these as opportunities to practice; addressing them independently will enhance your learning experience and prepare you for future challenges. However, you are **not required to** resolve intentional errors for credit, but it is highly recommended.

Python setup demo

```python
import numpy as np
import sympy as sp
from scipy.optimize import least_squares
from sympy.utilities.lambdify import lambdify

# Verification for NumPy
print("\nVerifying NumPy:")
arr = np.array([1, 2, 3, 4, 5])
print("Original Array:", arr)
arr_squared = np.square(arr)
print("Squared Array:", arr_squared)

# Verification for SymPy
print("\nVerifying SymPy:")
x = sp.Symbol('x')
expr = x**2 + 2*x + 1
derivative = sp.diff(expr, x)
integral = sp.integrate(expr, x)
print("Expression:", expr)
print("Derivative:", derivative)
print("Integral:", integral)

# Verification for SciPy
print("\nVerifying SciPy:")
def fun(x):
    return [x[0] + 0.5 * (x[0] - 3)**3 - 1.0]

result = least_squares(fun, [0.0])
print("Least Squares Result:", result)

# Verification for lambdify from SymPy
print("\nVerifying lambdify:")
func = lambdify(x, expr)
print("Evaluated Expression at x=2:", func(2))

print("\nVerification completed successfully.")
```

If you see the output without any errors, it means the required libraries (NumPy, SymPy, and SciPy) are installed and configured correctly in the "EE347Robotic_pyenv" virtual environment on both macOS and Windows operating systems. Additionally, `lambdify` from SymPy and `least_squares` from SciPy should be available as well.

At this point, you have successfully installed Python and configured the necessary libraries for numerical computing and symbolic mathematics within your "EE347Robotic_pyenv" virtual environment on both macOS and Windows.

Scientific computing in Python demo

In this section, we will explore three powerful libraries that form the backbone of numerical and symbolic computing: NumPy, SciPy, and SymPy. These tools will support the mathematical calculations you'll use throughout this course

NumPy - Numerical Computing with Python

NumPy is a fundamental library for numerical computing in Python, enabling efficient operations on large multi-dimensional arrays and matrices. It provides a wide range of mathematical functions to work with these arrays, making it an essential tool for scientific and engineering computations.

Demo code for NumPy:

```
import numpy as np

def test_numpy():
    # Test 1: Create a 1D NumPy array
    array_1d = np.array([1, 2, 3, 4, 5])
    assert np.array_equal(array_1d, np.array([1, 2, 3, 4, 5])), "1D array
creation failed"

    # Test 2: Create a 2D NumPy array
    array_2d = np.array([[1, 2], [3, 4]])
    assert np.array_equal(array_2d, np.array([[1, 2], [3, 4]])), "2D array
creation failed"

    # Test 3: Perform element-wise addition
    array_a = np.array([1, 2, 3])
    array_b = np.array([4, 5, 6])
    result_addition = array_a + array_b
    assert np.array_equal(result_addition, np.array([5, 7, 9])),
"Element-wise addition failed"

    # Test 4: Perform matrix multiplication
```

```
    matrix_a = np.array([[1, 2], [3, 4]])
    matrix_b = np.array([[5, 6], [7, 8]])
    result_multiply = np.dot(matrix_a, matrix_b)
    assert np.array_equal(result_multiply, np.array([[19, 22], [43, 50]])),
"Matrix multiplication failed"

    print("All NumPy tests passed!")

# Run the NumPy tests
test_numpy()
```

SciPy

SciPy builds on top of NumPy and provides additional numerical algorithms and functions for optimization, interpolation, integration, linear algebra, and more. It complements NumPy's functionality and expands the capabilities for scientific computing tasks.

Demo code for SciPy:

```
import numpy as np
from scipy.optimize import least_squares

def test_scipy_least_squares():
    # Test 1: Define a function for least squares optimization
    def func(x):
        return x[0] + 2 * x[1] - 3

    # Test 2: Provide initial guess and perform least squares optimization
    initial_guess = [1, 2]
    result = least_squares(func, initial_guess)

    # Test 3: Check the optimized values
    optimized_values = result.x
    expected_values = np.array([0.6, 1.2])
    assert np.allclose(optimized_values, expected_values), "Least squares
optimization failed"

    print("All SciPy (least_squares) tests passed!")

# Run the SciPy (least_squares) tests
```

```
test_scipy_least_squares()
```

SymPy - Symbolic Mathematics in Python

SymPy is a powerful library for symbolic mathematics in Python, allowing users to work with symbolic expressions, perform algebraic computations, and solve equations symbolically.

Demo code for SymPy:

```
import sympy as sp

def test_sympy():
    # Test 1: Create a symbolic variable
    x = sp.Symbol('x')
    assert str(x) == 'x', "Symbol creation failed"

    # Test 2: Create a symbolic expression
    expr = 2 * x + 3
    assert str(expr) == '2*x + 3', "Symbolic expression creation failed"

    # Test 3: Calculate the derivative of the expression
    derivative_expr = sp.diff(expr, x)
    assert str(derivative_expr) == '2', "Derivative calculation failed"

    # Test 4: Solve an equation
    equation = sp.Eq(expr, 10)
    solution = sp.solve(equation, x)
    assert solution == [7 / 2], "Equation solving failed"

    print("All SymPy tests passed!")

# Run the SymPy tests
test_sympy()
```

Kinematics Example

Kinematics is a crucial concept in robotics, allowing us to determine the joint angles required to position a robot's end-effector at a desired location and vice versa. The following demos show

how Python's scientific libraries can be used to solve forward and inverse kinematics. These codes are more complex than the earlier demos and may take some time to execute. Do not worry if every detail of the math is unfamiliar; we will revisit these concepts in future labs.

Kinematics demo code 1:

This first example demonstrates forward and inverse kinematics for a 2-link robotic arm. (hint: useful for Lab 2 and Lab 3):

```
import numpy as np
import sympy as sp
from scipy.optimize import least_squares

def test_kinematics():
   # Test 1: Forward Kinematics
   theta1 = np.pi / 4
   theta2 = np.pi / 3
   link1_length = 1.0
   link2_length = 0.8

   x_pos = link1_length * np.cos(theta1) + link2_length * np.cos(theta1 +
theta2)
   y_pos = link1_length * np.sin(theta1) + link2_length * np.sin(theta1 +
theta2)

   assert np.isclose(x_pos, 0.5000515451045311), "Forward kinematics - x
position calculation failed"
   assert np.isclose(y_pos, 1.4798474422178023), "Forward kinematics - y
position calculation failed"

   # Test 2: Inverse Kinematics
   x_desired = 1.0
   y_desired = 0.8

   theta1_sym = sp.Symbol('theta1')
   theta2_sym = sp.Symbol('theta2')

   x_eq = link1_length * sp.cos(theta1_sym) + link2_length *
sp.cos(theta1_sym + theta2_sym) - x_desired
   y_eq = link1_length * sp.sin(theta1_sym) + link2_length *
sp.sin(theta1_sym + theta2_sym) - y_desired
```

```
    solutions = sp.solve([x_eq, y_eq], [theta1_sym, theta2_sym])

    # Print and check all real solutions for inspection
    print("Inverse Kinematics Solutions:")
    valid_solution_found = False
    for sol in solutions:
        # Check if the solutions are real numbers
        if sol[0].is_real and sol[1].is_real:
            theta1_solution = float(sol[0].evalf())
            theta2_solution = float(sol[1].evalf())
            theta1_solution_deg = np.degrees(theta1_solution)
            theta2_solution_deg = np.degrees(theta2_solution)
            print(f"theta1: {theta1_solution_deg} degrees, theta2:
{theta2_solution_deg} degrees")


    # Test 3: Least Squares Optimization for Inverse Kinematics
    def inverse_kinematics_least_squares(x):
        theta1, theta2 = x
        return [
            link1_length * np.cos(theta1) + link2_length * np.cos(theta1 +
theta2) - x_desired,
            link1_length * np.sin(theta1) + link2_length * np.sin(theta1 +
theta2) - y_desired
        ]

    initial_guess = [0.5, 0.5]
    result = least_squares(inverse_kinematics_least_squares, initial_guess)

    theta1_optimized = np.degrees(result.x[0])
    theta2_optimized = np.degrees(result.x[1])

    print(f"Least Squares Optimized Solution: theta1: {theta1_optimized}
degrees, theta2: {theta2_optimized} degrees")

    print("All kinematics tests processed!")

# Run the kinematics tests
test_kinematics()
```

Kinematics demo code 2:

This second example demonstrates how to extend kinematics codes from a 2-link robotic arm into a 3-link robotic arm

```
import numpy as np
import sympy as sp
from scipy.optimize import least_squares

def test_kinematics():
    # Define link lengths
    link1_length = 1.0
    link2_length = 0.8
    link3_length = 0.6

    # Test 1: Forward Kinematics
    # Given joint angles (in radians)
    theta1 = np.pi / 4
    theta2 = np.pi / 3
    theta3 = np.pi / 6

    # Calculate the end-effector position (x, y) using forward kinematics
    x_pos = (link1_length * np.cos(theta1) +
             link2_length * np.cos(theta1 + theta2) +
             link3_length * np.cos(theta1 + theta2 + theta3))
    y_pos = (link1_length * np.sin(theta1) +
             link2_length * np.sin(theta1 + theta2) +
             link3_length * np.sin(theta1 + theta2 + theta3))

    print(f"Calculated Forward Kinematics Position: x = {x_pos}, y =
{y_pos}")

    # Expected values for forward kinematics (recomputed manually)
    expected_x = link1_length * np.cos(theta1) + link2_length *
np.cos(theta1 + theta2) + link3_length * np.cos(theta1 + theta2 + theta3)
    expected_y = link1_length * np.sin(theta1) + link2_length *
np.sin(theta1 + theta2) + link3_length * np.sin(theta1 + theta2 + theta3)

    print(f"Expected Forward Kinematics Position: x = {expected_x}, y =
{expected_y}")
```

```
    # Test the result against the expected values
    assert np.isclose(x_pos, expected_x), "Forward kinematics - x position
calculation failed"
    assert np.isclose(y_pos, expected_y), "Forward kinematics - y position
calculation failed"

    # Test 2: Inverse Kinematics
    # Given the desired end-effector position (x, y)
    x_desired = 1.0
    y_desired = 1.5
    phi_desired = np.deg2rad(20)

    # Create symbolic variables for joint angles
    theta1_sym = sp.Symbol('theta1')
    theta2_sym = sp.Symbol('theta2')
    theta3_sym = sp.Symbol('theta3')

    # Define the equations representing the end-effector position using
inverse kinematics
    x_eq = (link1_length * sp.cos(theta1_sym) +
            link2_length * sp.cos(theta1_sym + theta2_sym) +
            link3_length * sp.cos(theta1_sym + theta2_sym + theta3_sym) -
x_desired)
    y_eq = (link1_length * sp.sin(theta1_sym) +
            link2_length * sp.sin(theta1_sym + theta2_sym) +
            link3_length * sp.sin(theta1_sym + theta2_sym + theta3_sym) -
y_desired)

    phi_eq = (theta1_sym + theta2_sym + theta3_sym) - phi_desired

    # Solve the equations to find the joint angles (in radians)
    sol_vec = sp.nsolve([x_eq, y_eq, phi_eq],
                        [theta1_sym, theta2_sym, theta3_sym],
                        [0.5, 0.5, 0.5])
    solutions = [(float(sol_vec[0]), float(sol_vec[1]), float(sol_vec[2]))]

    # Print and check all real solutions for inspection
    print("Inverse Kinematics Solutions:")
    valid_solution_found = False
    for sol in solutions:
            theta1_solution, theta2_solution, theta3_solution = sol
            theta1_solution_deg = np.degrees(theta1_solution)
            theta2_solution_deg = np.degrees(theta2_solution)
```

```
            theta3_solution_deg = np.degrees(theta3_solution)
            print(f"theta1: {theta1_solution_deg} degrees, theta2:
{theta2_solution_deg} degrees, theta3: {theta3_solution_deg} degrees")

            # Check the solution against expected values
            # These expected values are placeholders; update them with your
actual expected values
            expected_theta1 = 30.0
            expected_theta2 = 45.0
            expected_theta3 = 60.0
            if np.isclose(theta1_solution_deg, expected_theta1) and
np.isclose(theta2_solution_deg, expected_theta2) and
np.isclose(theta3_solution_deg, expected_theta3):
                valid_solution_found = True

    if not valid_solution_found:
        print("No valid solution found that matches the expected values.")
    else:
        print("A valid solution matching the expected values was found.")

    # Test 3: Least Squares Optimization for Inverse Kinematics
    def inverse_kinematics_least_squares(x):
        theta1, theta2, theta3 = x
        return [
            link1_length * np.cos(theta1) +
            link2_length * np.cos(theta1 + theta2) +
            link3_length * np.cos(theta1 + theta2 + theta3) - x_desired,
            link1_length * np.sin(theta1) +
            link2_length * np.sin(theta1 + theta2) +
            link3_length * np.sin(theta1 + theta2 + theta3) - y_desired
        ]

    # Provide initial guess and perform least squares optimization
    initial_guess = [0.5, 0.5, 0.5]  # Initial guess for joint angles (in
radians)
    result = least_squares(inverse_kinematics_least_squares, initial_guess)

    theta1_optimized = np.degrees(result.x[0])
    theta2_optimized = np.degrees(result.x[1])
    theta3_optimized = np.degrees(result.x[2])

    print(f"Least Squares Optimized Solution: theta1: {theta1_optimized}
degrees, theta2: {theta2_optimized} degrees, theta3: {theta3_optimized}
```

```
degrees")

    print("All kinematics tests processed!")

# Run the kinematics tests
test_kinematics()
```

In this section, you've been introduced to three crucial libraries - NumPy, SciPy, and SymPy, each serving a distinct purpose in scientific computing with Python. You've also validated their functionalities through verification exercises, ensuring that they are ready to be utilized in your robotic and kinematics projects.

We also explored an Inverse Kinematics example, showcasing how we can use SymPy to solve the equations symbolically and SciPy's least squares optimization to find joint angles when the end-effector position is given.

The purpose of this exercise is to test if the libraries are correctly installed. Any runtime errors not caused by installation errors, can be ignored in this lab 1. You are, however, more than welcome to troubleshoot and solve these runtime errors. Understanding and applying inverse kinematics is fundamental in designing and controlling robotic systems, and Python's scientific computing libraries provide invaluable support for these complex computations. You will be using these skills and code for your Labs 2-4.

Introduction to Elephant Robotics

In Labs 2 and 3, you will be introduced to the robotic device: the Elephant Robotics Mecharm270Pi Robotic Arm. This document provides essential links to hardware documentation, detailed technical specifications for both the robotic arm and the rover, and access to additional resources. By the end of these labs, you will have a thorough understanding of these robotic devices, equipping you with the knowledge and skills to navigate the field of robotics confidently and lay a solid foundation for future explorations.

Grading Policy:

Your performance in EE 347 will be evaluated through a series of lab assignments designed to develop your skills and knowledge progressively. These lab assignments include written reports, technical deliverables, and video/photo submissions. Importantly, Lab 3 builds upon the content covered in Lab 2, which, in turn, builds upon the foundation established in Lab 1. Thus, consistent contributions and engagement throughout the quarter are vital to your success.

Lab Weightings:

Your total lab score in EE 347 will be determined through a weighted aggregation of your performance across Labs 1, 2, 3, and 4:

- Lab 1, 2, and Lab 3: Together, these account for 50% of your total lab grade. Each lab is equally weighted, and your performance in these labs will be assessed based:
    - Lab Document (30%)
    - Technical Objectives (50%)
    - Demo Questions (20%)
- Lab 4: Lab 4 carries the most significant weight, contributing 50% of your total lab grade. Specific grading policies for Lab 4 will be outlined in the Lab 4 manual.

Total Lab Score Calculation:

Lab 1: Points for Preparation

Lab 1 is a preparation lab to ensure you are ready for the future labs. Instead of a written report, the main task is to set up the programming environment and run the provided demos. Successfully completing all Lab 1 tasks will earn you 16.67% towards your final lab grade.

To receive credit, you must demonstrate completion through a **demo session** before the Lab 1 deadline.

Lab 1 Completion Verification:

You must demo Lab 1 to the teaching team before the Lab 1 deadline to receive credit. During your scheduled demo session, you will be asked to provide evidence of completing Lab 1. The primary verification requirements for Lab 1 include:

1. Installation of Preferred Python Editor: Demonstrate successful installation of the preferred Python editor as specified in the lab 1 instructions.

2. Configuration of EE 347 Environment: Verify the proper configuration of your EE 347 development environment, including the installation of necessary packages and dependencies, as outlined in the lab 1 guidelines.

3. Execution of Code with Correct Output: Showcase your ability to run the provided code within your configured environment and achieve the correct output as specified in the lab 1 instructions. This step validates that your environment setup is functioning as intended.

4. Complete the demo quiz on Canvas to answer some conceptual questions about Lab 1.

Lab 1 Grading:

Lab 1 will be graded based only on Live Demos and a demo quiz on Canvas:

- The Demonstrations will be graded as Complete, Attempted, or Incomplete and require a visual confirmation by an Instructor. **50 points total.**

Criteria	Full Credit (Complete)	Half Credit (Attempted)	No Credit (Incomplete)
Demo Session	• Successfully installs a Python editor (PyCharm recommended, or an alternative IDE) and demonstrates it during the demo. • Creates and configures the EE347Robotic_pyenv virtual environment with NumPy, SciPy, and SymPy correctly installed. • Runs the Python setup, NumPy, SciPy, SymPy, and Kinematics demo codes with correct output (minor runtime errors not related to installation are acceptable). • Student demonstrates completion of all tasks before the Lab 1 deadline.	• Partial installation or an IDE installed but not properly configured for the lab environment. • Environment created but missing one or more required libraries, or the setup is only partially functional. • Attempts to run code but results in errors due to partial setup or missing steps • Student attends demo but shows only partial completion.	• No installation demonstrated. • No virtual environment created or incorrectly configured. • No attempt to run demo code or environment does not run at all. • The student does not attend demo or provides no evidence of completion.

Lab Participation and Engagement:

Active participation and consistent effort in all labs, including Lab 1, are essential for achieving a successful outcome in each lab assignment. Your dedication and engagement in these labs will contribute to your overall learning experience and academic success in EE 347.

Consultation and Clarifications:

For full grading details and the specific requirements of each lab, please refer to the corresponding lab manuals. Feel free to communicate with your instructor about any questions

or concerns about the grading policy. Your instructors are here to assist you in your learning journey.

Your success in EE 347 hinges on your dedication and active participation in all aspects of the course, and we look forward to your contributions and achievements throughout the quarter.